



Universidade do Minho
Escola de Engenharia
Licenciatura de Engenharia Informática

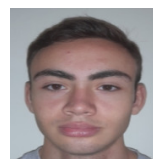
Programação Orientada a Objetos

Ano Letivo de 2025/2026
2º Semestre

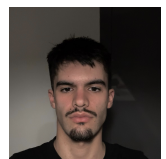
DomusControl



Maria Fernandes da Silva
A106802



Rafael Filipe Duarte de Andrade
A106918



Tomás Branco Dias
A107323

POO

10 de maio de 2026

Índice

1	Introdução	4
1.1	Contextualização do Projeto	4
1.2	Tema e Objetivos	4
1.3	O que se Pretende Alcançar	4
2	Metodologia de Desenvolvimento	5
2.1	Abordagem de Desenvolvimento	5
2.2	Organização da Aplicação	5
2.3	Princípios de Programação Orientada a Objetos	6
2.3.1	Encapsulamento	6
2.3.2	Herança	6
2.3.3	Polimorfismo	7
2.3.4	Abstração	7
2.3.5	Composição e Agregação	7
2.4	Padrões de Design Utilizados	8
2.4.1	Service Layer	8
2.4.2	Repository/DAO	8
2.4.3	Strategy	8
2.5	Decisões Arquiteturais	9
2.5.1	Separação entre Model e Service	9
2.5.2	Utilização de Serialização Binária	9
2.5.3	Exceções Personalizadas	10
2.6	Persistência de Dados	10
2.7	Tratamento de Erros	10
3	Descrição da Arquitetura de Classes	11
3.1	Estrutura Geral da Arquitetura	11
3.2	Principais Entidades do Sistema	11
3.2.1	Classe Casa	11
3.2.2	Classe Divisao	12
3.2.3	Classe Utilizador	12
3.2.4	Classe Dispositivo	13
3.3	Hierarquia de Dispositivos	13
3.4	Estrutura das Automações	14
3.5	Escalonamentos	14
3.6	Camada de Serviços	15
3.7	Persistência	15
3.8	Relógio Interno	15
3.9	Diagrama de classes	16
4	Descrição da Aplicação Desenvolvida	18
4.1	Inicialização da Aplicação	18

4.2	Gestão de Casas e Divisões	18
4.3	Gestão de Dispositivos	19
4.4	Automações	19
4.5	Cenários	20
4.6	Simulação Temporal	21
4.7	Persistência	21
4.8	Estatísticas	21
5	Testes e Validação	22
5.1	Teste de Criação de Entidades	22
5.2	Teste de Operação de Dispositivos	22
5.3	Teste de Automações	23
5.4	Teste de Persistência	23
5.5	Teste de Sugestões Automáticas	24
5.6	Teste de Tratamento de Erros	24
5.7	Validação Geral	24
6	Limitações e Trabalho Futuro	24
7	Conclusões	25

1 Introdução

1.1 Contextualização do Projeto

A automação doméstica é uma área em expansão contínua na indústria tecnológica contemporânea, refletindo a crescente necessidade de integração inteligente de sistemas no quotidiano das habitações. A implementação de soluções tecnológicas que permitam o controlo centralizado de dispositivos representa um passo significativo na otimização de recursos, conforto e segurança dos utilizadores. Neste contexto, surge o presente trabalho prático, que visa desenvolver uma aplicação simuladora de um sistema de automação doméstica, alinhado com plataformas existentes como o Home Assistant.

1.2 Tema e Objetivos

O projeto tem como tema central o desenvolvimento da aplicação DomusControl, uma plataforma de automação doméstica que integra funcionalidades avançadas de controlo de dispositivos inteligentes. Os objetivos específicos deste trabalho são:

- **Objetivo Pedagógico Principal:** Consolidar e aplicar na prática os conhecimentos fundamentais de programação orientada a objetos, incluindo encapsulamento, abstração de dados, herança, polimorfismo e princípios de design orientado a objetos.
- **Objetivo Funcional:** Desenvolver uma solução robusta que permita a modelação e gestão de entidades complexas (casas, divisões, dispositivos, utilizadores e regras de automação) de forma estruturada e escalável.
- **Objetivo Técnico:** Implementar mecanismos de persistência de dados, automações baseadas em gatilhos temporais e sensoriais, bem como funcionalidades de análise estatística da utilização do sistema.

1.3 O que se Pretende Alcançar

Ao término deste projeto, espera-se ter desenvolvido uma aplicação completa e funcional que demonstre:

1. **Modelação orientada a objetos:** Uma arquitetura bem definida com classes, relações e responsabilidades claramente estabelecidas.
2. **Funcionalidades integradas:** Um sistema capaz de gerir múltiplos dispositivos em diversos cenários, permitindo operações individuais sobre dispositivos, gestão de automações e simulação de diferentes cenários de utilização doméstica.

3. Qualidade técnica: Código legível, bem documentado, seguindo boas práticas de programação e estruturado segundo as normas da orientação a objetos.
4. Aplicabilidade prática: Uma solução que, embora simuladora, demonstre os princípios que sustentam sistemas reais de automação doméstica, validando assim a compreensão e aplicação dos conceitos teóricos lecionados.

2 Metodologia de Desenvolvimento

2.1 Abordagem de Desenvolvimento

O desenvolvimento da aplicação DomusControl foi realizado segundo uma abordagem modular e iterativa, permitindo dividir o sistema em componentes independentes e facilitar a integração progressiva das funcionalidades.

O projeto foi estruturado em diferentes módulos de responsabilidade específica, possibilitando que os elementos do grupo trabalhassem de forma paralela sobre diferentes componentes da aplicação. Esta estratégia permitiu melhorar a organização do código, reduzir conflitos durante o desenvolvimento e facilitar os testes de integração.

A divisão principal do sistema foi realizada da seguinte forma:

Módulo	Componentes	Responsabilidades
Modelo de Dados	Casas, divisões, dispositivos, automações, utilizadores	Representação das entidades do domínio
Serviços	CasaService, DispositivoService, AutomacaoService, EstatisticaService	Lógica de negócio e processamento das operações
Persistência e Interface	BaseDados, RepositorioBinario, menus e consola	Armazenamento de dados e interação com o utilizador

A organização modular permitiu construir uma solução mais extensível e coerente, facilitando a manutenção e futura evolução do sistema.

2.2 Organização da Aplicação

A aplicação foi organizada em packages especializados, promovendo separação de responsabilidades e desacoplamento entre componentes.

Os principais packages desenvolvidos foram:

- app – contém a classe principal responsável pelo arranque da aplicação;

- `model` – inclui as entidades centrais do domínio, como casas, divisões, dispositivos, automações e utilizadores;
- `service` – responsável pela lógica de negócio e coordenação das operações;
- `ui` – contém os menus e mecanismos de interação em modo texto;
- `persistence` – implementa os mecanismos de gravação e carregamento do estado da aplicação;
- `exception` – contém as exceções específicas utilizadas no tratamento de erros.

Esta organização segue uma abordagem em camadas inspirada em princípios de separação de responsabilidades semelhantes ao modelo MVC.

2.3 Princípios de Programação Orientada a Objetos

2.3.1 Encapsulamento

O encapsulamento foi aplicado através da utilização de atributos privados e métodos públicos controlados.

As entidades da aplicação mantêm o seu estado interno protegido, sendo as alterações efetuadas apenas através de métodos específicos que garantem validação e consistência dos dados.

Por exemplo, os dispositivos apenas podem alterar o seu estado através de operações como `ligar()` ou `desligar()`, evitando alterações diretas aos atributos internos.

2.3.2 Herança

A herança foi utilizada para representar relações do tipo “é um” entre entidades do sistema.

A classe base `Dispositivo` define comportamento comum a todos os dispositivos inteligentes da aplicação, sendo posteriormente especializada por subclasses como:

- `Lampada;`
- `ArCondicionado;`
- `Cortina;`
- `ColunaDeSom;`
- `PortaoGaragem;`
- `TomadaInteligente;`

- Sensor.

A utilização de herança permitiu reutilizar código comum e simplificar a extensão futura da aplicação com novos dispositivos.

2.3.3 Polimorfismo

O polimorfismo foi utilizado para permitir que diferentes dispositivos fossem tratados de forma uniforme.

O sistema consegue operar sobre coleções de objetos do tipo `Dispositivo` sem necessidade de conhecer o tipo concreto de cada elemento. Desta forma, métodos como `ligar()` ou `desligar()` executam implementações distintas dependendo do tipo concreto do objeto em tempo de execução.

O mesmo princípio foi aplicado às automações, onde diferentes ações implementam comportamentos específicos através do método `executar()`.

2.3.4 Abstração

A abstração foi aplicada através de classes base, interfaces e enumerações.

Classes como:

- `Dispositivo`;
- `Acao`;
- `Gatilho`;

definem contratos genéricos especializados posteriormente pelas subclasses.

As enumerações foram utilizadas para representar categorias fixas do sistema, como:

- `TipoSensor`;
- `TipoDivisao`;
- `ModoArCondicionado`.

2.3.5 Composição e Agregação

A estrutura do sistema utiliza relações hierárquicas entre entidades.

Composição

- Uma Casa contém várias Divisao;
- Uma Divisao contém vários Dispositivo;
- Uma Automacao contém Gatilho e Acao.

Agregação

- Um Utilizador pode estar associado a múltiplas casas;
- As automações referenciam dispositivos sem controlar o seu ciclo de vida.

2.4 Padrões de Design Utilizados

2.4.1 Service Layer

A lógica de negócio foi centralizada em serviços especializados como:

- CasaService;
- DispositivoService;
- AutomacaoService;
- EstatisticaService.

Esta abordagem permitiu separar operações de negócio das entidades e da interface da aplicação.

2.4.2 Repository/DAO

A persistência foi implementada através de um mecanismo semelhante ao padrão Repository.

As classes RepositorioBinario e BaseDados encapsulam as operações de leitura e escrita, desacoplando a lógica de persistência do restante sistema.

2.4.3 Strategy

As diferentes ações executadas pelas automações foram modeladas através de estratégias distintas.

Classes como:

- AcaoLigar;

- `AcaoDesligar`;
- `AcaoAjustarCortina`;

implementam comportamentos diferentes mantendo a mesma estrutura base.

2.5 Decisões Arquiteturais

2.5.1 Separação entre Model e Service

A aplicação foi estruturada com separação entre entidades do domínio e lógica de negócio.

O package `model` contém apenas as entidades principais do sistema, enquanto o package `service` centraliza operações, validações e regras de negócio.

Esta abordagem permitiu:

- reduzir acoplamento;
- evitar duplicação de código;
- facilitar manutenção;
- simplificar testes;
- permitir futura integração com outros tipos de interface.

2.5.2 Utilização de Serialização Binária

A persistência do sistema foi implementada através de serialização binária utilizando a interface `Serializable`.

Esta solução foi escolhida por:

- permitir guardar objetos complexos de forma simples;
- preservar automaticamente relações entre entidades;
- facilitar carregamento completo do estado da aplicação;
- reduzir complexidade relativamente à utilização de bases de dados externas.

Tendo em conta o contexto académico do projeto, esta abordagem revelou-se adequada e suficientemente robusta.

2.5.3 Exceções Personalizadas

Foi criada uma hierarquia própria de exceções baseada na classe `DomusException`.

A utilização de exceções específicas permitiu:

- identificar erros de forma mais precisa;
- melhorar a legibilidade do código;
- separar diferentes tipos de falhas;
- fornecer mensagens mais claras ao utilizador.

Exceções como `CasaNaoEncontradaException` OU `DispositivoNaoEncontradoException` permitem tratar erros específicos sem recorrer apenas às exceções genéricas da linguagem Java.

2.6 Persistência de Dados

A aplicação suporta gravação e carregamento do estado completo do sistema.

O mecanismo de persistência permite armazenar:

- casas;
- divisões;
- dispositivos;
- utilizadores;
- automações;
- escalonamentos;
- registos de interação.

A informação é serializada para ficheiros binários e posteriormente recuperada sem perda das relações entre objetos.

2.7 Tratamento de Erros

Foi implementado um sistema hierárquico de exceções específicas para aumentar a robustez da aplicação.

Entre as exceções desenvolvidas destacam-se:

- `CasaNaoEncontradaException`;

- `DispositivoNaoEncontradoException;`
- `DivisaoNaoEncontradaException;`
- `UtilizadorNaoEncontradoException;`
- `PersistenciaException.`

Este mecanismo permite tratar erros de forma estruturada e melhorar a experiência de utilização da aplicação.

3 Descrição da Arquitetura de Classes

A arquitetura do sistema DomusControl foi desenvolvida segundo uma estrutura modular orientada aos objetos, permitindo representar as diferentes entidades do domínio de forma organizada e extensível.

O sistema encontra-se dividido em diferentes componentes responsáveis pela modelação das entidades, execução da lógica de negócio, persistência de dados e interação com o utilizador.

3.1 Estrutura Geral da Arquitetura

A arquitetura principal da aplicação encontra-se organizada em quatro camadas fundamentais:

- entidades do domínio;
- serviços;
- persistência;
- interface com o utilizador.

As entidades representam os objetos centrais do sistema, enquanto os serviços coordenam as operações e regras de negócio. A camada de persistência é responsável pela gravação e recuperação de dados, sendo a interação realizada através de menus em modo texto.

3.2 Principais Entidades do Sistema

3.2.1 Classe Casa

A classe `Casa` representa uma habitação gerida pelo sistema.

Principais atributos:

- identificador;
- nome;
- morada;
- lista de divisões;
- lista de proprietários;
- lista de usufrutuários.

A classe é responsável por organizar divisões e utilizadores associados à habitação.

3.2.2 Classe Divisao

A classe `Divisao` representa uma divisão pertencente a uma casa.

Principais atributos:

- identificador;
- nome;
- tipo de divisão;
- lista de dispositivos.

Cada divisão agrega dispositivos inteligentes associados ao respetivo espaço físico.

3.2.3 Classe Utilizador

A classe `Utilizador` representa os utilizadores da aplicação.

Principais atributos:

- identificador;
- nome;
- email;
- password.

Um utilizador pode estar associado a múltiplas casas com diferentes permissões.

3.2.4 Classe Dispositivo

A classe `Dispositivo` funciona como superclasse para todos os dispositivos inteligentes do sistema.

Principais atributos:

- identificador;
- marca;
- modelo;
- consumo por hora;
- estado de funcionamento;
- número de ativações;
- tempo total ligado.

Esta classe centraliza comportamento comum a todos os dispositivos implementados.

3.3 Hierarquia de Dispositivos

A arquitetura foi desenvolvida de forma a permitir extensão futura através da criação de novos dispositivos.

Atualmente encontram-se implementadas as seguintes subclasses:

- `Lampada`;
- `ArCondicionado`;
- `Cortina`;
- `PortaoGaragem`;
- `ColunaDeSom`;
- `TomadaInteligente`;
- `Sensor`.

Cada subclasse adiciona características específicas ao comportamento herdado da superclasse `Dispositivo`.

Por exemplo:

- as lâmpadas suportam intensidade luminosa e temperatura de cor;

- o ar condicionado suporta diferentes modos de funcionamento;
- as cortinas e portões possuem percentagem de abertura;
- as colunas permitem controlo de volume;
- os sensores armazenam valores associados ao tipo de sensor.

3.4 Estrutura das Automações

O sistema de automações foi modelado através das entidades `Automacao`, `Gatilho` e `Acao`.

Uma automação é composta por:

- um gatilho;
- uma ação;
- informação de ativação;
- associação a uma casa.

Os gatilhos representam condições de execução automática, estando atualmente implementados:

- `GatilhoHorario`;
- `GatilhoSensor`.

As ações representam operações executadas automaticamente sobre dispositivos:

- `AcaoLigar`;
- `AcaoDesligar`;
- `AcaoAjustarCortina`.

3.5 Escalonamentos

Os escalonamentos representam operações dependentes do tempo.

Cada escalonamento possui:

- identificador;
- horário associado;
- ação a executar;

- estado de ativação.

Esta funcionalidade permite simular comportamentos automáticos baseados em horários definidos pelo utilizador.

3.6 Camada de Serviços

A camada de serviços centraliza a lógica de negócio da aplicação.

Os principais serviços implementados foram:

- CasaService;
- DispositivoService;
- AutomacaoService;
- EstatisticaService;
- CenarioService;
- SugestaoService.

Os serviços comunicam com a camada de persistência e coordenam operações sobre as entidades do sistema.

3.7 Persistência

A persistência foi implementada através das classes:

- BaseDados;
- RepositorioBinario.

A classe BaseDados funciona como ponto central de acesso aos dados da aplicação, enquanto o repositório encapsula as operações de leitura e escrita em ficheiros binários.

3.8 Relógio Interno

O sistema possui uma classe Relogio responsável pela simulação temporal da aplicação.

Esta entidade permite:

- avançar o tempo manualmente;
- testar automações temporais;

- validar escalonamentos.

A utilização de um relógio interno permite desacoplar a aplicação do relógio real do sistema operativo.

3.9 Diagrama de classes

As figuras seguintes apresentam o diagrama de classes principal da aplicação DomusControl, ilustrando as relações entre as diferentes entidades do sistema, bem como a organização modular da arquitetura desenvolvida.

O diagrama encontra-se dividido em três partes principais:

- estrutura das entidades centrais do domínio;
- sistema de automações e serviços;
- hierarquia de exceções.

A arquitetura evidencia a utilização de herança na hierarquia de dispositivos, composição entre casas, divisões e dispositivos, bem como a separação entre camadas de modelo, serviços e persistência.

A relação entre *Casa* e *Divisao* representa uma composição, dado que as divisões existem apenas no contexto de uma casa. De forma semelhante, as divisões agregam dispositivos inteligentes associados ao espaço físico correspondente.

As automações são compostas por gatilhos e ações, permitindo representar regras dinâmicas de funcionamento do sistema. Já a camada de serviços centraliza a lógica de negócio, comunicando com a camada de persistência através da classe *BaseDados*.

Por fim, o sistema de exceções personalizadas permite um tratamento estruturado de erros, aumentando a robustez e previsibilidade da aplicação.

A decomposição do diagrama em múltiplas imagens teve como objetivo melhorar a legibilidade da representação UML, permitindo observar com maior detalhe as relações entre entidades, serviços e mecanismos de tratamento de erros.

O diagrama de classes foi desenvolvido com o objetivo de representar graficamente as relações estruturais entre as diferentes classes da aplicação.

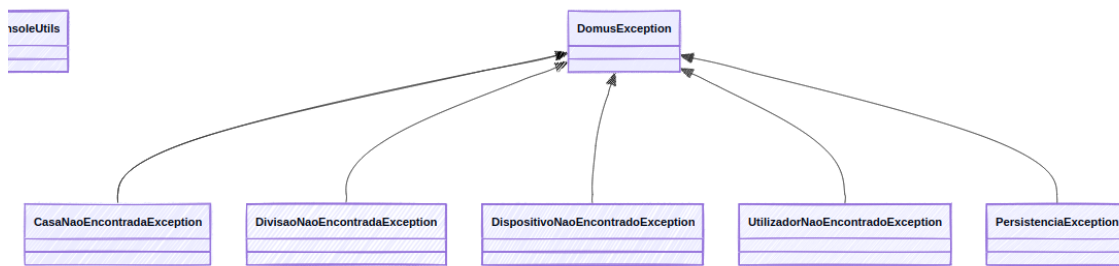


Figura 3: Hierarquia de exceções

4 Descrição da Aplicação Desenvolvida

A aplicação DomusControl foi desenvolvida como uma plataforma de simulação de automação doméstica, permitindo gerir casas inteligentes através de uma interface em modo texto.

O sistema permite criar e administrar entidades como casas, divisões, dispositivos, utilizadores e automações, possibilitando simular funcionalidades típicas de soluções reais de domótica.

4.1 Inicialização da Aplicação

Ao iniciar o programa, o utilizador tem acesso ao menu principal da aplicação, através do qual pode navegar pelas diferentes funcionalidades disponíveis.

O sistema suporta:

- carregamento de dados previamente guardados;
- criação manual de entidades;
- utilização de dados de demonstração;
- gravação do estado da aplicação.

4.2 Gestão de Casas e Divisões

Uma das funcionalidades centrais consiste na gestão de casas e respetivas divisões.

O utilizador administrador pode:

- criar novas casas;
- adicionar divisões;
- associar dispositivos às divisões;
- consultar informação da instalação.

Cada casa funciona como uma estrutura hierárquica composta por múltiplas divisões, permitindo uma organização semelhante à de uma habitação real.

4.3 Gestão de Dispositivos

A aplicação suporta vários tipos de dispositivos inteligentes.

Entre os dispositivos implementados destacam-se:

- lâmpadas;
- cortinas;
- tomadas inteligentes;
- colunas de som;
- sensores;
- portões de garagem;
- ar condicionado.

Cada dispositivo possui características específicas.

Por exemplo:

- as lâmpadas permitem ajustar intensidade luminosa;
- as cortinas permitem definir o grau de abertura;
- as colunas permitem controlar o volume;
- o ar condicionado suporta diferentes modos de funcionamento.

Os dispositivos podem ser operados individualmente através da interface da aplicação.

4.4 Automações

O sistema implementa mecanismos de automação baseados em gatilhos.

As automações permitem executar ações automaticamente quando determinadas condições são verificadas.

Foram implementados dois tipos principais de gatilhos:

- gatilhos temporais;
- gatilhos baseados em sensores.

Exemplos de automações suportadas:

- ligar luzes automaticamente;
- desligar dispositivos;
- ajustar cortinas;
- executar ações em horários definidos.

As automações podem atuar sobre múltiplos dispositivos de forma automática.

4.5 Cenários

Além das automações e escalonamentos, o sistema DomusControl suporta a definição de cenários personalizados.

Um cenário representa um conjunto de ações executadas simultaneamente sobre múltiplos dispositivos da habitação, permitindo adaptar rapidamente o estado da casa a diferentes situações do quotidiano.

Foram implementados diversos cenários inspirados em funcionalidades reais de sistemas de automação doméstica, entre os quais:

- **Sair de Casa** – desliga luzes, fecha cortinas e desativa dispositivos não essenciais;
- **Jantar com Amigos** – ativa iluminação ambiente e dispositivos sociais;
- **Deitar** – desliga dispositivos ativos e reduz atividade da habitação;
- **Acordar** – abre cortinas e ativa dispositivos previamente configurados.

Os cenários podem ser executados manualmente pelo utilizador através da interface da aplicação, permitindo atuar sobre vários dispositivos de forma rápida e eficiente.

Esta funcionalidade aproxima o comportamento da aplicação dos sistemas reais de domótica, onde diferentes contextos de utilização podem ser ativados através de perfis previamente definidos.

4.6 Simulação Temporal

Como o projeto representa um ambiente de simulação, foi implementado um sistema interno de relógio.

A classe `Relogio` permite:

- avançar o tempo manualmente;
- testar automações temporais;
- simular funcionamento contínuo do sistema.

Este mecanismo permite validar regras dependentes de horário sem utilizar o relógio real do sistema operativo.

4.7 Persistência

A aplicação permite guardar e recuperar o estado completo do sistema.

Desta forma:

- casas;
- divisões;
- dispositivos;
- utilizadores;
- automações;
- escalonamentos;
- registos de interação;

podem ser recuperados posteriormente sem perda de informação.

4.8 Estatísticas

Foi igualmente implementado um módulo estatístico responsável pela análise da utilização do sistema.

Entre as funcionalidades disponíveis destacam-se:

- identificação da casa com maior consumo;
- dispositivos mais utilizados;

- divisões com maior número de dispositivos;
- análise de ativações.

Estas estatísticas permitem obter indicadores relevantes sobre o comportamento da instalação.

5 Testes e Validação

A validação do sistema foi realizada através de diversos cenários de teste funcionais, permitindo confirmar o correto funcionamento das principais funcionalidades implementadas.

5.1 Teste de Criação de Entidades

Foram realizados testes de criação de:

- utilizadores;
- casas;
- divisões;
- dispositivos.

Os testes confirmaram:

- associação correta entre entidades;
- armazenamento adequado dos dados;
- funcionamento das relações hierárquicas.

5.2 Teste de Operação de Dispositivos

Foram testadas operações individuais sobre diferentes dispositivos.

Exemplos:

- ligar e desligar lâmpadas;
- alterar intensidade luminosa;
- ajustar abertura de cortinas;
- controlar volume de colunas;

- alterar modos do ar condicionado.

Os resultados demonstraram comportamento consistente e adequado ao tipo de dispositivo.

5.3 Teste de Automações

As automações foram testadas utilizando diferentes gatilhos.

Teste de gatilho temporal Foi criada uma automação para ligar uma lâmpada num horário específico.

Resultado:

- ao avançar o relógio interno para a hora definida, a automação foi executada corretamente.

Teste de gatilho por sensor Foi simulada uma condição de sensor associada ao fecho automático de cortinas.

Resultado:

- a automação executou corretamente a ação configurada.

5.4 Teste de Persistência

Foram realizados testes de gravação e carregamento de dados.

Procedimento:

1. criação de entidades;
2. gravação em ficheiro binário;
3. encerramento da aplicação;
4. carregamento do ficheiro guardado.

Resultado:

- os dados foram recuperados corretamente;
- as relações entre objetos foram preservadas.

5.5 Teste de Sugestões Automáticas

Foram simuladas interações repetidas sobre dispositivos em horários semelhantes.

Resultado:

- o sistema identificou padrões recorrentes;
- foram geradas sugestões automáticas de escalonamentos.

5.6 Teste de Tratamento de Erros

Foram simuladas situações inválidas:

- procura de dispositivos inexistentes;
- acesso a divisões inexistentes;
- carregamento de ficheiros inválidos.

O sistema respondeu adequadamente através das exceções específicas implementadas.

5.7 Validação Geral

Os testes realizados permitiram validar:

- funcionamento da arquitetura orientada aos objetos;
- integração entre módulos;
- robustez da persistência;
- execução correta das automações;
- estabilidade geral da aplicação.

Os resultados demonstram que a aplicação cumpre os principais requisitos definidos no enunciado do projeto.

6 Limitações e Trabalho Futuro

Apesar de cumprir os principais requisitos definidos no enunciado, a aplicação desenvolvida apresenta algumas limitações inerentes ao âmbito académico do projeto.

Uma das principais limitações consiste na utilização de uma interface exclusivamente em modo texto, o que reduz a componente visual e a facilidade de utilização relativamente a soluções comerciais de automação doméstica.

A persistência de dados foi implementada através de serialização binária, não existindo integração com sistemas de bases de dados relacionais ou não relacionais. Embora adequada para o contexto do projeto, esta solução apresenta menor flexibilidade e escalabilidade.

Outra limitação prende-se com a ausência de comunicação com dispositivos físicos reais, funcionando toda a aplicação num ambiente de simulação controlado.

Relativamente às automações inteligentes previstas no requisito 8.4 do enunciado, foi implementado um mecanismo simplificado de sugestão automática de escalonamentos baseado na repetição de padrões de utilização registados no histórico de interações do sistema.

Embora esta abordagem permita identificar comportamentos recorrentes e sugerir automatizações básicas, não foi implementado um sistema avançado de aprendizagem automática ou inteligência artificial.

Como trabalho futuro, poderiam ser acrescentadas diversas melhorias:

- desenvolvimento de interface gráfica;
- integração com dispositivos IoT reais;
- utilização de bases de dados;
- sistema de notificações;
- autenticação mais robusta;
- automações inteligentes baseadas em análise comportamental.

Estas melhorias permitiriam aproximar a aplicação de plataformas reais de automação doméstica.

7 Conclusões

O desenvolvimento da aplicação DomusControl permitiu aplicar de forma prática os principais conceitos da programação orientada a objetos através da construção de um sistema modular de automação doméstica.

Ao longo do projeto foram implementadas funcionalidades de gestão de casas, divisões, dispositivos, utilizadores, automações, escalonamentos e cenários, permitindo simular o comportamento de uma plataforma de domótica semelhante às existentes no mercado.

A arquitetura desenvolvida procurou respeitar os princípios fundamentais da orientação aos objetos, nomeadamente encapsulamento, abstração, herança e polimorfismo. A separação em diferentes camadas e packages contribuiu para uma solução organizada, extensível e de fácil manutenção.

Durante o desenvolvimento surgiram diversos desafios técnicos, principalmente relacionados com:

- modelação das relações entre entidades;
- implementação das automações;
- preservação das relações entre objetos na persistência;
- integração dos diferentes módulos desenvolvidos pelos elementos do grupo.

A utilização de serialização binária, serviços especializados e exceções personalizadas permitiu construir uma aplicação mais robusta e coerente do ponto de vista arquitetural.

Os testes realizados demonstraram que o sistema cumpre os principais requisitos definidos no enunciado, incluindo gestão de entidades, execução de automações, persistência de dados e produção de estatísticas de utilização.

Relativamente ao requisito 8.4 do enunciado, referente à geração automática de automações e escalonamentos, foi implementado um mecanismo de análise simples baseado no histórico de interações do utilizador, permitindo sugerir automaticamente escalonamentos a partir de padrões repetidos de utilização.

Em termos globais, o projeto constituiu uma experiência relevante para consolidar conhecimentos de engenharia de software e desenvolvimento orientado aos objetos, permitindo compreender melhor os desafios associados à construção de aplicações modulares e evolutivas.

Apesar das limitações identificadas, o projeto permitiu desenvolver uma solução funcional, modular e extensível, capaz de demonstrar de forma prática os conceitos fundamentais da programação orientada a objetos. O resultado obtido evidencia não apenas a implementação das funcionalidades previstas no enunciado, mas também a aplicação consistente de princípios de engenharia de software e de programação orientada a objetos na construção de uma solução modular, robusta e extensível.